

Rabbit R1: From dm-verity corruption to Ubuntu Touch

A practical textbook on MediaTek boot modes, Android Verified Boot, A/B slots, dynamic partitions, recovery, and the supported UBports path

Edition: August 7, 2026 **Target device:** rabbit r1 (r1, MediaTek MT6765 / Helio P35) **Starting point:** bootloader now unlocks successfully; stock boot/vbmeta* images have been flashed in several combinations; fastbootd has been unreliable; the device reports or has reported dm-verity corruption.

Scope and risk This is a device-recovery and OS-installation guide, not a generic “run these commands blindly” recipe. Several commands erase user data or rewrite boot-critical partitions. Read the whole recovery chapter before executing the destructive blocks. Keep the bootloader **unlocked** throughout the Ubuntu Touch path. Never relock while modified or verification-disabled images are installed.

Contents

1. The short answer
2. What the Rabbit R1 actually is
3. The boot chain: from power-on to Android/Linux userspace
4. Android Verified Boot, AVB, vbmeta, and dm-verity
5. Why “dm-verity corruption” can persist
6. A/B slots and why slot state matters
7. super, dynamic partitions, bootloader fastboot, and fastbootd
8. MediaTek BROM, preloader, mtkclient, and mtkbootcmd.py
9. What the FRP edit did in r1_escape
10. Reading your terminal history as a state machine
11. Host-side problems in your log: Python, sudo, USB, and mke2fs
12. What state your R1 is probably in now
13. Recovery strategy: establish a coherent stock base first
14. Non-destructive inspection checklist
15. Manual stock recovery path for your v0.8.293 files
16. Slot recovery and first stock boot
17. Last-resort R1-specific seccfg dm-verity recovery
18. Why you do **not** need to port Ubuntu Touch yourself
19. How Ubuntu Touch on the R1 is architected
20. Installing Ubuntu Touch with the UBports Installer
21. Why the Ubuntu Touch path does not depend on your failing fastbootd step
22. After Ubuntu Touch boots
23. Troubleshooting matrix
24. Command reference

25. Glossary

26. Sources and further reading

1. The short answer

Your logs show that the most difficult prerequisite has already been crossed: you changed the R1's unlock permission state, entered bootloader fastboot, and `fastboot flashing unlock` returned OKAY. After that, writes to `boot_a`, `boot_b`, and the AVB metadata partitions were accepted.

The correct route from here is **not** to keep following the `r1_escape` AOSP/CipherOS flow and it is **not** to build a new Ubuntu Touch port. The rabbit r1 already has a UBports device target and an installer configuration. UBports lists the R1 as an installer-supported device and the installer config names the device `r1`.^[1]^[2]

The practical sequence is:

1. Stop mixing stock images, `r1_escape`'s disabled `vbmeta`, and unrelated custom-ROM instructions.
2. Inspect the current slot and lock state without writing anything.
3. Restore one coherent **stock rabbitOS v0.8** boot chain and `super` image while keeping the bootloader unlocked.
4. Boot stock once. If stock still produces `dm-verity` corruption, diagnose the R1-specific persistent `seccfg` state described by the community recovery write-up; do not jump to this step before a clean stock test.^[3]
5. Once stock v0.8 boots and the bootloader is unlocked, run the UBports Installer as a normal user, select **rabbit r1**, leave **Bootstrap** enabled, select the current stable channel offered by the installer, and wipe userdata when switching from Android.
6. Let the installer flash its own `vbmeta`, `boot`, and complete `super` image, boot UBports recovery, and install the Ubuntu Touch system image.^[2]
7. After the first successful Ubuntu Touch boot, update through **System Settings -> Updates**. Rabbit R1 is included in the Ubuntu Touch 24.04 support list.^[4]

A critical distinction: **the supported R1 Ubuntu Touch bootstrap flashes a complete `super` image from bootloader fastboot. It does not require you to make the `fastboot reboot-fastboot` step from the CipherOS workflow work first.**^[2]

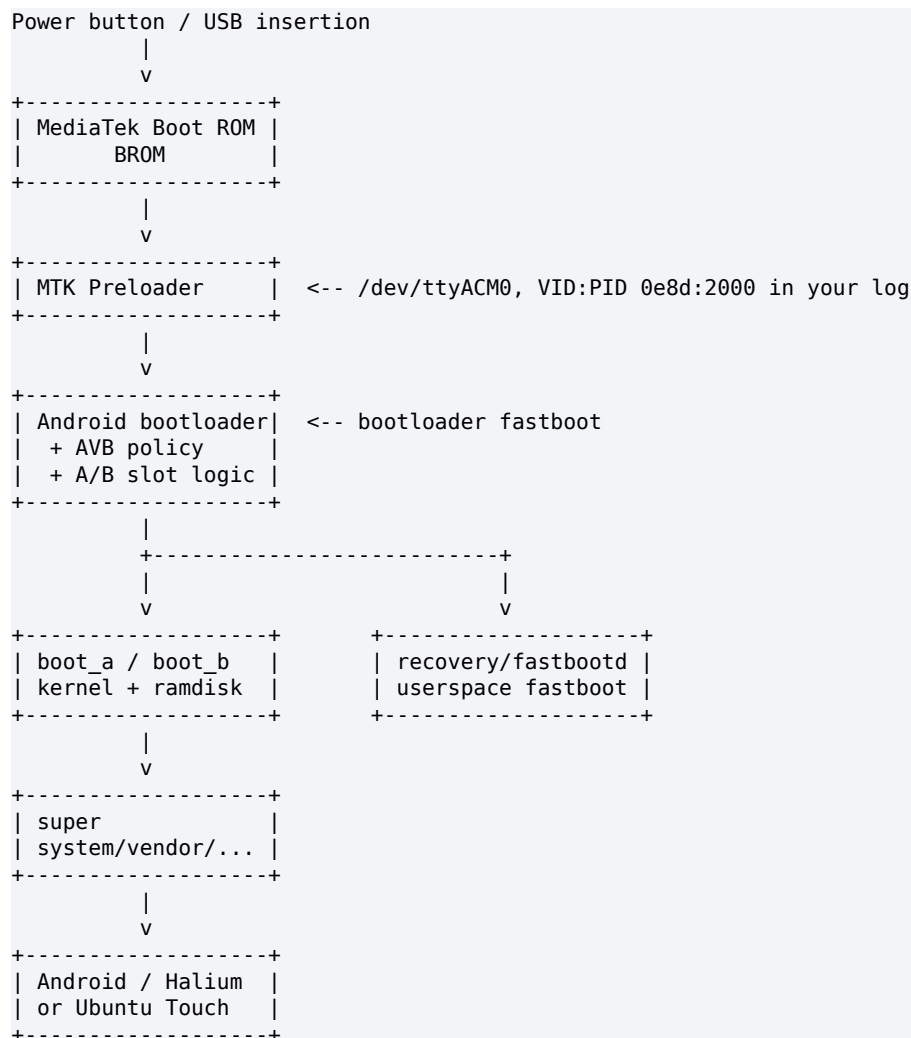
2. What the Rabbit R1 actually is

For recovery purposes, treat the R1 as a small MediaTek Android device with unusual controls, not as a fundamentally new computer architecture. The UBports device database identifies it as an ARM64 device based on the MediaTek Helio P35 family, with Android/rabbitOS as the vendor base.[1]

This matters because nearly every concept in your terminal log comes from the Android/MediaTek boot stack:

- MediaTek **BROM** and **preloader** run before Android's normal bootloader interface.
- Android's bootloader exposes **fastboot**.
- Android Verified Boot (**AVB**) authenticates the partitions that make up the operating system.
- Large read-only filesystems are protected at runtime by **dm-verity**.
- The device uses **A/B slot metadata**, so bootability is not determined only by the contents of `boot.img`.
- Android's `system`, `vendor`, and related logical partitions live inside a physical **super** partition.
- Ubuntu Touch on this device reuses the Android hardware enablement through **Halium** instead of replacing every vendor component with mainline Linux drivers.

A useful mental model is:



3. The boot chain: from power-on to userspace

A “boot failure” can occur at several distinct layers. The symptoms look similar from the outside, but the fixes are different.

3.1 BROM and preloader

MediaTek chips contain immutable boot ROM code. The preloader is the next vendor-specific stage. In your log, `mtkclient` recognized the chip as MT6765/MT8768t and reported the target as unprotected. That is why `mtkclient` could upload its download-agent stages and read/write raw eMMC partitions.

This layer is below normal Android fastboot. It is the reason you can recover a device that cannot boot Android at all.

3.2 Android bootloader

The Android bootloader performs several jobs:

- expose bootloader fastboot;
- decide which A/B slot to boot;
- load `boot`, device-tree, and other boot-critical images;
- enforce Android Verified Boot policy;
- pass boot parameters such as the slot suffix and dm-verity mode into the kernel.

The orange unlocked-device warning belongs to this layer. An unlocked bootloader changes trust policy; it does not, by itself, guarantee that the images form a coherent system.

3.3 Kernel and early userspace

`boot.img` contains the Linux kernel and ramdisk components needed for early boot. A correct `boot.img` can still fail if its expected vendor/system partitions do not match, if AVB descriptors are inconsistent, or if the bootloader has marked the slot unbootable.

3.4 The Android logical partitions

Modern Android devices group logical partitions such as `system`, `vendor`, `product`, and others inside `super`. Android calls these **dynamic partitions**.^[5]

This is the layer that a custom Android GSI, CipherOS, or the R1 Ubuntu Touch bootstrap changes most dramatically.

4. Android Verified Boot, AVB, vbmeta, and dm-verity

The words in your error message are often confused because several integrity mechanisms work together.

4.1 Verified Boot is the overall trust chain

Android Verified Boot aims to ensure that executable code and read-only system data come from an expected, authenticated build. Android 8 and newer use AVB 2.0 as the reference implementation.[6]

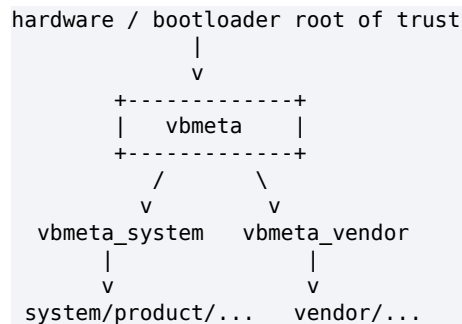
Small boot-critical partitions can be verified by hashing the entire image before execution. Large filesystem partitions are generally verified using hash trees while blocks are read.[7]

4.2 What vbmeta is

vbmeta is metadata that describes what should be trusted and how. It may contain or chain to descriptors for other partitions. On the R1 you have seen:

```
vbmeta_a / vbmeta_b
vbmeta_system_a / vbmeta_system_b
vbmeta_vendor_a / vbmeta_vendor_b
```

Conceptually:



Flashing a stock `boot.img` while leaving modified or mismatched `vbmeta` metadata can still produce a boot failure. Conversely, flashing verification-disabled `vbmeta` while later relocking the bootloader creates a different trust contradiction: a locked device expects a valid trusted chain but sees content intended for an unlocked development state.

4.3 What `--disable-verity` `--disable-verification` actually means

When fastboot is asked to flash a `vbmeta` image with these options, it modifies AVB flags in the image it sends. In your log, fastboot explicitly printed:

```
Rewriting vbmeta struct at offset: 0
```

That is evidence that the data written was **not byte-for-byte the original stock vbmeta image**, even though the input filename was the stock file.

This can be useful for development, but it is a poor diagnostic state when trying to answer “does clean stock AVB boot correctly?” For that test, flash the original stock `vbmeta*` files **without** the disable flags while leaving the bootloader unlocked.

4.4 What dm-verity is

`dm-verity` is a Linux device-mapper target that checks filesystem blocks against a cryptographic hash tree as they are read. Android uses it for large verified partitions.[8]

A simplified tree:



Change a data block and the chain to the trusted root no longer matches.

5. Why dm-verity corruption can persist

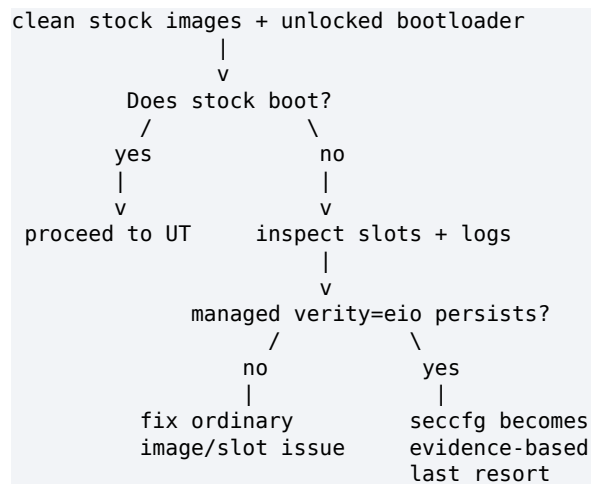
AOSP documents a managed dm-verity error flow in which the bootloader can switch the next boot into `eio` mode after a verification failure. In `eio` mode, corrupted reads return I/O errors rather than immediately restarting; the bootloader is expected to return to normal mode after a new valid OS is installed.[7]

The Rabbit R1 community recovery write-up reports a device-specific complication: on at least the investigated R1 build, a dm-verity error state remained persisted in the MediaTek `seccfg` partition even after stock images were restored. The bootloader continued passing:

```
androidboot.veritymode=eio
androidboot.veritymode.managed=yes
```

and recreated the managed-verity state until the relevant authenticated `seccfg` header state and persistent AVB record were corrected.[3]

That does **not** mean every R1 showing dm-verity corruption needs a raw `seccfg` write. The correct diagnostic order is:



6. A/B slots and why slot state matters

A/B devices keep two bootable sets of slot-aware partitions. The active slot can fail several times and eventually be marked unbootable. Android's boot-control design exposes variables such as `current-slot`, `slot-successful`, `slot-unbootable`, and `slot-retry-count` through fastboot.[9]

The important point is that **correct bytes are not sufficient**. A slot with valid images may still be skipped if metadata says it is unbootable.

Typical inspection:

```
fastboot getvar current-slot
fastboot getvar slot-count
fastboot getvar slot-successful:a
fastboot getvar slot-unbootable:a
fastboot getvar slot-retry-count:a
fastboot getvar slot-successful:b
fastboot getvar slot-unbootable:b
fastboot getvar slot-retry-count:b
```

AOSP specifies that setting a slot active should clear its unbootable state and reset its retry count.[10]

Therefore the first recovery action for bad slot metadata is usually:

```
fastboot set_active a
```

and then inspect the variables again. The R1 community recovery notes additionally describe erasing `boot_para` and `para` when both slots had been “burned down” and ordinary slot reset behavior was insufficient. Treat that as a device-specific escalation, not as the first thing to do.[3]

7. **super**, dynamic partitions, bootloader fastboot, and fastbootd

This distinction explains a large portion of your confusion.

7.1 **super** is a physical container

Android dynamic partitions place logical system, vendor, product, and related partitions inside a physical super partition.[5]

7.2 Bootloader fastboot versus fastbootd

Bootloader fastboot runs in the bootloader. **fastbootd** is a userspace fastboot implementation reached through the recovery/userspace boot path. AOSP introduced it so Android could safely manipulate dynamic logical partitions.[11]

Check which one you are in:

```
fastboot getvar is-userspace
```

Interpretation:

```
is-userspace: no    -> bootloader fastboot
is-userspace: yes   -> fastbootd
```

7.3 Why your **fastboot** **reboot-fastboot** hangs

AOSP describes `reboot fastboot` as a transition into userspace fastboot/recovery.[11] If the device's recovery/userspace boot path is broken, mismatched, or unable to complete AVB/slot boot, the host can print:

```
< waiting for any device >
```

because the bootloader fastboot USB interface disappeared but fastbootd never appeared.

This is exactly different from “fastboot itself is broken”: you repeatedly proved that bootloader fastboot works after sending FASTBOOT through the MediaTek preloader.

7.4 Why this is not a blocker for the supported Ubuntu Touch installer

The current UBports r1 installer configuration downloads a Rabbit-R1-specific `boot.img`, a packaged `super.img`, and a `vbmeta.img`; in the bootstrap step it flashes the **physical** ``vbmeta``, ``boot``, and ``super`` **partitions** from bootloader fastboot, then reboots into UBports recovery for the system-image installation.[2]

In other words:

```
CipherOS / generic system.img path
bootloader fastboot -> fastbootd -> flash logical system
```

```
Ubuntu Touch R1 installer path
bootloader fastboot -> flash whole super -> UBports recovery -> system-image install
```

Do not spend hours fixing fastbootd just because an unrelated custom-ROM guide needed it.

8. MediaTek BROM, preloader, `mtkclient`, and `mtkbootcmd.py`

Your R1 exposes several USB personalities during recovery. In your own log, `mtkbootcmd.py` found:

```
MT65xx Preloader - CDC ACM Communication Interface
VID:PID = 0E8D:2000
/dev/ttyACM0
```

and wrote the ASCII string:

```
FASTBOOT
```

The script is not “flashing fastboot.” It is asking the running MediaTek preloader to branch into the Android bootloader's fastboot mode.

`mtkclient` is lower-level. It talks to BROM/preloader, uploads MediaTek download-agent code, enumerates eMMC, and can read/write named partitions even when Android cannot boot.

That is why this pair is so useful:

```
mtkclient      -> raw partition access / last-resort recovery
mtkbootcmd.py  -> convenient preloader -> bootloader-fastboot transition
fastboot       -> normal unlocked bootloader flashing
```

For routine Ubuntu Touch installation, prefer normal fastboot and the UBports installer. Keep `mtkclient` as a recovery tool rather than the default installation mechanism.

9. What the FRP edit did in `r1_escape`

The current `r1_escape` script documents this sequence:[12]

1. read the `frp` partition;
2. examine the final byte;
3. change a final `0x00` to `0x01`;
4. write `frp` back;
5. enter fastboot;
6. run `fastboot flashing unlock`.

Your terminal history demonstrates the effect directly:

```
before FRP edit:
fastboot flashing get_unlock_ability
unlock_ability = 0

fastboot flashing unlock
Unlock operation is not allowed
```

After the FRP edit and reconnect:

```
fastboot flashing unlock
Start unlock flow
OKAY
```

So in this workflow the FRP edit is not “disabling dm-verity.” It is altering the device state that gates whether the bootloader unlock flow is permitted.

This distinction matters because three independent concepts were changing at once:

Concept	Your action	Effect
Unlock permission	FRP final-byte edit	Allowed <code>fastboot flashing unlock</code>
Bootloader lock state	<code>fastboot flashing unlock</code>	Allowed protected partition writes
AVB enforcement metadata	<code>flashing vbmeta</code> with disable flags	Changed verification behavior

Treat them as separate switches.

10. Reading your terminal history as a state machine

This chapter converts the long shell log into a smaller sequence of state transitions.

10.1 Initial fastboot command injection worked, but USB fastboot did not appear

You installed `pyserial`, then:

```
python3 r1_escape/mtkbootcmd.py FASTBOOT
...
b'FASTBOOT' cmd sent
```

but `fastboot` devices initially returned nothing. That means “the serial command was written” and “the host successfully enumerated the next fastboot USB personality” were two separate events. Later runs did enumerate fastboot correctly.

10.2 Stock boot flashing failed because the device was locked

You tried:

```
fastboot flash boot_a ../boot.img
```

and got:

```
FAILED (remote: 'not allowed in locked state')
```

This is normal bootloader policy, not a bad `boot.img`.

10.3 The bootloader explicitly told you unlocking was disabled

You then saw:

```
unlock_ability = 0
Unlock operation is not allowed
```

This was the central blocker that `r1_escape` addresses.

10.4 `sudo python3` used a different Python environment

You installed `pyusb` under your `pyenv` Python:

```
/home/manuel/.pyenv/versions/3.13.2/...
```

but then ran:

```
sudo python3 mtk ...
```

which used the system/root Python environment and failed with:

```
ModuleNotFoundError: No module named 'usb'
```

When you ran `python3 mtk ...` without `sudo`, the `pyenv` interpreter could import `PyUSB` and `mtkclient` worked.

This is a host Python environment issue, not a MediaTek issue.

10.5 The FRP edit succeeded

`mtkclient` dumped `frp`, you changed its final byte, and `mtkclient` wrote it back. After sending `FASTBOOT` again, the host saw:

```
919109A491600019115B fastboot
```

Then:

```
fastboot flashing unlock
OKAY
```

The bootloader was now unlocked.

10.6 `fastboot -w` only partially succeeded

Your output was:

```
Erasing 'userdata' OKAY
/usr/lib/android-sdk/platform-tools/mke2fs failed with status 1
fastboot: error: Cannot generate image for userdata
```

This has two stages:

1. the device accepted the **erase**;
2. the host failed to generate a fresh filesystem image for the subsequent **format** operation.

Android's fastboot build distributes filesystem-generation utilities such as `mke2fs` alongside fastboot because formatting can involve creating a filesystem image on the host before it is sent to the device. [13]

So the useful conclusion is not “userdata could not be erased.” It is “the erase happened, but this host fastboot package cannot currently finish the format step.”

For the stock-recovery test, your firmware package already includes `userdata.img`, so you can avoid relying on `fastboot -w`. For Ubuntu Touch, use a current supported UBports Installer and, if formatting still fails, fix/update the host platform-tools rather than repeatedly erasing the device.

10.7 You flashed several different `vbmeta` states

The history includes all of these patterns:

- `r1_escape/vbmeta.img` with disable flags;
- stock `vbmeta_a` and `vbmeta_b` with disable flags;
- stock `vbmeta_system_*` and `vbmeta_vendor_*`;
- later another flash of `r1_escape/vbmeta.img`.

This makes the current AVB state difficult to infer from filenames. A “clean stock test” must overwrite this ambiguity with one known set.

10.8 You restored stock `boot_a` and `boot_b`

These writes succeeded after unlocking. That is good, but `boot` is only one piece of the boot chain. You did not show a corresponding clean stock flash of the complete `super` filesystem in the provided history.

10.9 The fastbootd transition kept failing

After `fastboot reboot-fastboot`, bootloader fastboot disappeared and userspace fastboot did not enumerate. This is compatible with a broken userspace/recovery boot path. It does not invalidate the working preloader -> bootloader-fastboot path.

11. Host-side problems in your log: Python, `sudo`, USB, and `mke2fs`

Before doing more device writes, reduce host-side ambiguity.

11.1 Use one Python interpreter deliberately

From the R1 working directory:

```
cd /home/manuel/code/others/rabbit-r1/r1_escape
python3 -m venv .venv-r1
. .venv-r1/bin/activate
python -m pip install --upgrade pip
python -m pip install pyserial
python -m pip install -r mtkclient/requirements.txt
```

Confirm:

```
which python
python -V
python -c 'import serial, usb.core; print("serial+usb OK")'
```

If a command truly must run as root, invoke the same interpreter explicitly rather than silently switching Python installations:

```
sudo "$PWD/.venv-r1/bin/python" mtkclient/mtk --help
```

Prefer working udev permissions and unprivileged execution when possible; it avoids root-owned dumps and Python-environment mismatches.

11.2 Understand `cdc_acm`

For your `mtkbootcmd.py` method, the MediaTek preloader appears as a CDC ACM serial device (`/dev/ttyACM0`), so the `cdc_acm` driver is useful at that moment.

Some R1 custom-ROM notes remove `cdc_acm` later because of a device-specific USB-interface conflict with ADB. Do not generalize that into “`cdc_acm` must always be off.” Think in phases:

```
Need preloader serial (/dev/ttyACM0) -> cdc_acm loaded
Need ordinary fastboot                -> unrelated to ttyACM serial driver
Need R1 ADB after a custom ROM        -> follow that ROM's USB guidance
```

11.3 Stop using `fastboot -w` as a diagnostic loop

The command is destructive and your host package has already shown that its formatting helper fails. Repeating it does not tell you anything new.

Use the stock `userdata.img` during stock recovery, or fix/update the host fastboot stack before relying on format operations.

12. What state your R1 is probably in now

Based strictly on the provided history, the strongest conclusions are:

Known:

- the FRP unlock gate was changed successfully;
- fastboot flashing unlock now succeeds;
- protected partition flashing is allowed;
- stock boot.img was written to both boot_a and boot_b at least once;
- stock AVB metadata was written to both slots, but top-level vbmeta was at times modified with disable flags;
- r1_escape's own vbmeta.img was also flashed later;
- userdata was erased at least once, but host-side reformatting failed;
- bootloader fastboot is recoverable through mtkbootcmd.py;
- userspace fastboot/fastbootd is not reliably reached.

Unknown from the history:

- which slot is currently active;
- whether either slot is marked unbootable;
- the exact contents of the current physical super partition;
- the current top-level vbmeta flags on the active slot;
- whether seccfg still contains a persistent managed-verification error state;
- whether stock rabbitOS can boot if given one coherent, fully restored stock set.

This is why the next step should be **inspection**, not another guess-flash.

13. Recovery strategy: establish a coherent stock base first

The UBports R1 installer asks users to begin from stock rabbitOS v0.8 and an unlocked bootloader.[1][2]

That requirement is technically sensible: the Ubuntu Touch port still depends on the device's Android-era firmware/vendor hardware support. A random mixture of CipherOS, AOSP GSI, stock boot images, and disabled AVB metadata is not a reliable base.

Use this escalation ladder:

```

Level 0  Non-destructive inspection
      |
      v
Level 1  Clean stock v0.8 boot chain + super, bootloader stays unlocked
      |
      +---- stock boots -> go directly to UBports Installer
      |
      v
Level 2  Repair A/B slot metadata if necessary
      |
      +---- stock boots -> go to UBports Installer
      |
      v
Level 3  Capture evidence of persistent managed dm-verity=eio
      |
      v
Level 4  seccfg recovery (raw MTK write; last resort)
      |
      +---- stock boots -> go to UBports Installer

```

Do not relock the bootloader anywhere in this ladder.

14. Non-destructive inspection checklist

Start from your current working directory:

```
cd /home/manuel/code/others/rabbit-r1
```

Enter bootloader fastboot using the method that already works:

```
python3 r1_escape/mtkbootcmd.py FASTBOOT
```

Wait for:

```
fastboot devices -l
```

Then save a diagnostic snapshot:

```
mkdir -p diagnostics
{
  date -Is
  fastboot --version
  fastboot devices -l
  fastboot getvar product
  fastboot getvar current-slot
  fastboot getvar slot-count
  fastboot getvar is-userspace
  fastboot getvar slot-successful:a
  fastboot getvar slot-unbootable:a
  fastboot getvar slot-retry-count:a
  fastboot getvar slot-successful:b
  fastboot getvar slot-unbootable:b
  fastboot getvar slot-retry-count:b
  fastboot flashing get_unlock_ability
  fastboot getvar all
} 2>&1 | tee diagnostics/fastboot-state.txt
```

Do not worry if one vendor variable returns “unknown variable”; keep the rest.

What you want to see

At minimum:

```
fastboot device is present
bootloader is still unlocked
current-slot is a or b
at least one slot is not unbootable
is-userspace: no
```

If both slots report unbootable, go to Chapter 16 after the stock flash rather than before it.

15. Manual stock recovery path for your v0.8.293 files

15.1 Preferred method: official Rabbit flashing path

The community de-verity recovery notes explicitly recommend official Rabbit firmware/flashing tools as the starting point and reserve raw MediaTek seccfg work for last-resort recovery.[3]

If the official Rabbit flasher accepts your device in its current state, use it. Keep the bootloader unlocked afterward if your next goal is Ubuntu Touch.

15.2 Manual path matching the files you already extracted

You extracted:

```
rabbit_OS_v0.8.293_20250516110545/
```

Set a shell variable:

```
cd /home/manuel/code/others/rabbit-r1
FW="$PWD/rabbit_OS_v0.8.293_20250516110545"
```

Check the files before flashing:

```
ls -lh \
"$FW/boot.img" \
"$FW/dtbo.img" \
"$FW/lk.img" \
"$FW/tee.img" \
"$FW/md1img.img" \
"$FW/spmfw.img" \
"$FW/scp.img" \
"$FW/sspm.img" \
"$FW/gz.img" \
"$FW/vbmeta.img" \
"$FW/vbmeta_system.img" \
"$FW/vbmeta_vendor.img" \
"$FW/super.img" \
"$FW/userdata.img"
```

If any named file is absent, stop and adapt to the exact official package rather than substituting a similarly named file from another guide.

15.3 Enter bootloader fastboot

```
python3 r1_escape/mtkbootcmd.py FASTBOOT
fastboot devices -l
```

Confirm the bootloader is unlocked by attempting only read-only queries first. Do **not** run fastboot flashing lock.

15.4 Restore the slot-aware stock boot chain

The goal here is coherence, not de-verification. Flash the stock AVB images **without** `--disable-verity` and **without** `--disable-verification`.

```
fastboot flash mdlimb_a "$FW/mdlimb.img"
fastboot flash spmfw_a "$FW/spmfw.img"
fastboot flash scp_a "$FW/scp.img"
fastboot flash sspm_a "$FW/sspm.img"
fastboot flash gz_a "$FW/gz.img"
fastboot flash lk_a "$FW/lk.img"
fastboot flash boot_a "$FW/boot.img"
fastboot flash dtbo_a "$FW/dtbo.img"
fastboot flash tee_a "$FW/tee.img"
fastboot flash vbmeta_a "$FW/vbmeta.img"
fastboot flash vbmeta_system_a "$FW/vbmeta_system.img"
fastboot flash vbmeta_vendor_a "$FW/vbmeta_vendor.img"
```

Mirror the supplied stock boot-chain images to slot B as a recovery measure:

```
fastboot flash mdlimb_b "$FW/mdlimb.img"
fastboot flash spmfw_b "$FW/spmfw.img"
fastboot flash scp_b "$FW/scp.img"
fastboot flash sspm_b "$FW/sspm.img"
fastboot flash gz_b "$FW/gz.img"
fastboot flash lk_b "$FW/lk.img"
fastboot flash boot_b "$FW/boot.img"
fastboot flash dtbo_b "$FW/dtbo.img"
fastboot flash tee_b "$FW/tee.img"
fastboot flash vbmeta_b "$FW/vbmeta.img"
fastboot flash vbmeta_system_b "$FW/vbmeta_system.img"
fastboot flash vbmeta_vendor_b "$FW/vbmeta_vendor.img"
```

If `logo.bin` is present and you want the full stock presentation layer restored:

```
fastboot flash logo "$FW/logo.bin"
```

15.5 Restore the stock `super` filesystem

Your package contains `super.img`. Flash the physical `super` partition:

```
fastboot flash super "$FW/super.img"
```

This is different from `fastboot flash system system.img`: you are replacing the dynamic-partition container as a whole, which bootloader fastboot can support.

15.6 Restore userdata without relying on your broken host `mke2fs` path

Because `fastboot -w` has already failed at host-side filesystem creation, use the stock userdata image for this recovery test:

```
fastboot flash userdata "$FW/userdata.img"
fastboot erase metadata
```

This is destructive to personal data, but your earlier unlock/wipe attempts have already made data preservation unrealistic.

Do not erase FRP for the Ubuntu Touch goal

The generic factory-recovery write-up includes an FRP erase as part of returning a device to factory behavior. Your objective is different: the FRP modification is what made bootloader unlocking possible. Preserve your current FRP and keep a backup rather than erasing it unnecessarily.

16. Slot recovery and first stock boot

After the coherent stock flash:

```
fastboot set_active a
```

Inspect:

```
fastboot getvar current-slot
fastboot getvar slot-retry-count:a
fastboot getvar slot-unbootable:a
fastboot getvar slot-successful:a
fastboot getvar slot-retry-count:b
fastboot getvar slot-unbootable:b
fastboot getvar slot-successful:b
```

If slot A is no longer marked unbootable, try:

```
fastboot reboot
```

Expected behavior for an unlocked but otherwise stock device:

1. an unlocked/orange-state style warning may appear;
2. the device proceeds into stock boot;
3. first boot may take longer than a normal reboot because userdata was replaced.

16.1 If both slots remain unbootable

The R1 community recovery notes used this device-specific reset after both slot retry counters had been exhausted:[3]

```
fastboot erase boot_para
fastboot erase para
fastboot reboot bootloader
```

Re-enter/catch fastboot, then:

```
fastboot set_active a
```

Re-check slot variables before booting.

Use this only when the slot variables show that ordinary `set_active` did not recover bootability.

16.2 If stock boots successfully

Stop recovering. Do not “improve” the stock state by flashing disabled vbmeta again. You have reached the correct base for the Ubuntu Touch installer:

```
stock rabbit05 v0.8 base
+
bootloader unlocked
+
working bootloader fastboot
```

Proceed to Chapter 20.

17. Last-resort R1-specific `seccfg` dm-verity recovery

This chapter is deliberately separated from the normal path.

17.1 When to consider it

Consider `seccfg` repair only if all of these are true:

- you restored a coherent stock boot chain and stock super;
- you kept the bootloader unlocked;
- the slot metadata is bootable;
- stock still shows or loops on dm-verity corruption;
- fresh boot diagnostics indicate that the bootloader is still forcing managed verity into `eio` mode.

The community write-up's crucial evidence was the boot parameter pair:

```
androidboot.veritymode=eio
androidboot.veritymode.managed=yes
```

persisting after a stock flash.[3]

17.2 Capture evidence first

Use `mtkclient` to read diagnostic partitions rather than immediately writing `seccfg`. For example, after reproducing one failed boot, reconnect to preloader/BROM and dump `expdb`:

```
cd /home/manuel/code/others/rabbit-r1/r1_escape/mtkclient
python3 mtk r expdb /tmp/r1-expdb.bin
strings -a /tmp/r1-expdb.bin | grep -Ei 'verity|avb|slot|boot' | tail -n 200
```

Also back up `seccfg` and FRP before any last-resort write:

```
python3 mtk r seccfg /tmp/r1-seccfg-before.bin
python3 mtk r frp /tmp/r1-frp-before.bin
sha256sum /tmp/r1-seccfg-before.bin /tmp/r1-frp-before.bin
```

Keep copies somewhere outside the working directory.

17.3 What the community recovery found

For the investigated R1 `seccfg` V4 format, the write-up identifies:

- `lock_state` at offset `0x0c`; value 3 represented the unlocked state in that case;
- a critical/dm-verity state word at offset `0x10`; the bad state was 1 and the recovered state was 0;
- a persistent managed-verity record in the range `0x240..0x2bf`.

The important warning is that the V4 header is authenticated. **Do not treat `seccfg` as an ordinary plaintext structure and hand-edit one integer in a hex editor.** The recovery procedure used `mtkclient`'s `seccfg` implementation to generate a valid authenticated header, then combined that generated header with the full partition and cleared the persistent managed-verity record.[3]

17.4 Safe conceptual procedure

The evidence-based last-resort flow is:

1. backup full `seccfg`
2. patch/use `mtkclient` so an "unlock" candidate is generated with:


```
lock_state = unlocked
critical/dm-verity state = 0
```
3. capture the generated authenticated header before direct DA write
4. copy that authenticated header into a full `seccfg` backup
5. zero only the known managed-verity record region
6. verify the resulting bytes and file size
7. write the complete corrected `seccfg`
8. stop sending commands immediately if the device begins a real boot

The community repository contains the exact patch strategy and should be treated as the authoritative device-specific reference for this escalation.[3]

17.5 Why this is a last resort

A wrong `seccfg` write is qualitatively different from flashing `boot.img`. You are modifying MediaTek security/configuration state outside the normal Android OS partitions. That is recoverable in some scenarios because BROM/preloader access exists, but it is also easier to make the device harder to recover.

Do not perform this step merely because the screen once printed “dm-verity corruption.” Perform it because a clean stock test plus fresh diagnostics show the persistent R1-specific state described above.

18. Why you do not need to port Ubuntu Touch yourself

Older Ubuntu Touch advice often starts with “build Haliu for your device.” That is correct for an unsupported device. It is no longer the right starting point for the R1.

The current UBports device page lists:

- device codename r1;
- a Haliu-based R1 target;
- installer availability;
- stock rabbitOS v0.8 as the required starting firmware;
- an unlocked bootloader as a prerequisite.[1]

The UBports infrastructure metadata also identifies a Rabbit R1 community port and device/kernel source locations.[14]

Ubuntu Touch 24.04-1.0 explicitly included the Rabbit R1 in its supported-device list.[4]

So your job is now an **installation/recovery job**, not a new-port engineering project.

19. How Ubuntu Touch on the R1 is architected

Ubuntu Touch is Linux, but on Android-derived devices it is not equivalent to taking an Ubuntu Desktop ARM image and writing it to storage.

UBports explains a modern port as a combination of:[15]

- the Ubuntu Touch root filesystem;
- Halium components, including the device kernel/system-side hardware adaptation;
- vendor blobs from the Android base.

Conceptually:

```

+-----+
| Ubuntu Touch userspace          |
| Lomiri shell, system services, apps, Ubuntu rootfs |
+-----+
| Halium / libhybris / Android compatibility layer  |
+-----+
| Android vendor services and proprietary blobs     |
+-----+
| R1 Linux kernel + device drivers                  |
+-----+
| MT6765 hardware                                   |
+-----+

```

This explains why the installer cares about the Android/rabbitOS firmware family: the proprietary firmware and hardware interfaces remain part of the usable device stack.

It also explains why “Ubuntu Touch” is not identical to “desktop Ubuntu on a tiny screen.” Ubuntu Touch uses OTA system images and generally keeps its root filesystem read-only; UBports documents Libertine/other mechanisms for installing conventional desktop packages without treating the base OS like a normal mutable Debian workstation.[16]

20. Installing Ubuntu Touch with the UBports Installer

Once stock v0.8 boots correctly and the bootloader remains unlocked, stop using the `r1_escape` flashing script.

20.1 Prerequisites

You want this state:

```
[ ] rabbit r1 hardware confirmed
[ ] stock rabbitOS v0.8 base restored
[ ] bootloader unlocked
[ ] bootloader fastboot visible to the host
[ ] no unresolved persistent stock dm-verity loop
[ ] USB cable stable
[ ] host fastboot stack functional
```

The current R1 installer config requires an installer compatible with version `>=0.9.2-beta`; use a current UBports Installer release rather than an old package.[2]

20.2 Run the installer as your normal user

UBports documentation explicitly advises **not** to run the installer with `sudo`, because root execution can create permission problems in its cache.[17]

Start the UBports Installer normally.

20.3 Select the device

If automatic detection is unreliable because you are not booted into Android, manually select:

```
rabbit r1
codename: r1
```

Confirm the installer warnings about model, stock v0.8 firmware, and unlocked bootloader.[2]

20.4 Choose the install options

For the first switch from Android/rabbitOS to Ubuntu Touch:

- choose the **stable channel currently offered by the installer**;
- enable **Wipe Userdata**;
- leave **Bootstrap** enabled.

Do not choose a development channel just because a blog post mentions a newer release candidate. Your first goal is a known-good boot.

20.5 Enter bootloader fastboot when asked

The current R1 config describes a hardware route using the scroll wheel and power button.[2]

If that is awkward and your proven preloader method works, you can place the device in bootloader fastboot yourself:

```
python3 /home/manuel/code/others/rabbit-r1/r1_escape/mtkbootcmd.py FASTB00T
fastboot devices -l
```

Then let the installer continue.

20.6 What the installer actually flashes

The R1 bootstrap currently downloads:

- an R1-specific `boot.img`;
- an R1-specific packaged `super.img`;

- a vbmeta.img used by the installation bootstrap.

It verifies checksums, unpacks the super archive, and performs the equivalent high-level sequence:[2]

```

bootloader fastboot
|
+--> flash vbmeta
+--> flash boot
+--> flash super
+--> format userdata (when wipe selected)
|
v
reboot to UBports recovery
|
v
system-image installation
|
v
reboot / finish

```

This is why you should **not pre-flash** r1_escape/vbmeta.img immediately before running the installer. Let the installer establish the image set it expects.

20.7 If userdata formatting fails again

If the UBports Installer reports a host-side format failure similar to your /usr/lib/android-sdk/platform-tools/mke2fs error:

1. save the installer log;
2. update/fix the host Android platform-tools stack instead of repeatedly wiping;
3. re-run the installer as a normal user;
4. do not add random manual fastboot -w cycles between installer attempts.

Your earlier message proves the device can erase userdata; the problem was in the host filesystem-generation stage.

21. Why the Ubuntu Touch path does not depend on your failing fastbootd step

This deserves a direct answer because it is the main place your earlier workflow went sideways.

r1_escape is designed to install an Android AOSP userdebug `system.img`. Its script therefore:

```
unlock
wipe
flash disabled vbmeta
reboot to fastbootd
flash logical system
```

That makes sense for a dynamic logical `system` partition.[12][11]

The R1 Ubuntu Touch installer, by contrast, flashes a complete `super` image in its bootstrap.[2]

Therefore:

- **do not continue debugging `fastboot reboot-fastboot` as the next required Ubuntu Touch step;**
- first fix the stock/verity state;
- then use the supported installer bootstrap.

Fastbootd may still be useful for Android ROM work, but it is no longer the critical path to your stated goal.

22. After Ubuntu Touch boots

22.1 Update through the supported mechanism

Ubuntu Touch 24.04-1.0 lists Rabbit R1 as a supported device. UBports' normal upgrade path for existing devices is through **System Settings -> Update**.^[4]

Because channel metadata and release rollouts change over time, do not hard-code an old channel string from a forum guide. Use the stable update offered to the device.

22.2 Expect Ubuntu Touch behavior, not desktop Ubuntu behavior

The Ubuntu Touch root filesystem is designed around OTA image updates and is normally read-only. Conventional desktop packages are not managed exactly as on an Ubuntu laptop; UBports documents containers such as Libertine for traditional desktop applications.^[16]

22.3 Keep the bootloader unlocked

The installed Ubuntu Touch boot chain is not the original locked Rabbit trust chain. Relocking after installation can put you back into a “correct signature”/bad-state failure.

22.4 Keep your recovery assets

Archive these on another machine:

```
rabbitOS v0.8.293 firmware archive
original FRP backup (if you have one)
current FRP backup
pre-repair seccfg backup (if created)
mtkclient version/commit used
r1_escape mtkbootcmd.py
UBports Installer logs from the successful installation
```

That turns a future failure from archaeology into a reproducible recovery.

23. Troubleshooting matrix

Symptom	Layer	Likely meaning	Next action
fastboot devices empty immediately after FASTBOOT serial write	USB transition	preloader command sent, fastboot USB personality not yet enumerated	reconnect/reset; verify no process owns tty; retry proven mtkbootcmd.py flow
not allowed in locked state	bootloader policy	bootloader locked	do not flash; confirm unlock path/state
unlock_ability = 0	unlock gate	bootloader refuses unlock request	FRP/OEM-unlock gate not enabled; your r1_escape FRP step solved this previously
fastboot flashing unlock returns OKAY	bootloader policy	unlock complete	proceed; expect data wipe/security warning
mke2fs failed with status 1 after Erasing 'userdata' OKAY	host fastboot tools	erase succeeded; host format image generation failed	stop repeating -w; use stock userdata for recovery; update host platform-tools for installer
fastboot reboot-fastboot -> waiting forever	userspace/recovery boot	bootloader fastboot exited, fastbootd never enumerated	not a UT blocker; recover stock, then use UBports bootstrap
dm-verity corruption after mixed custom flashes	AVB/dm-verity	image/metadata mismatch or persisted managed error	first restore coherent stock boot chain + super
stock images restored but veritymode=eio persists	R1 security state	evidence for seccfg managed-verity persistence	back up, then follow R1 de-verity recovery last-resort procedure
both slots unbootable: yes	A/B metadata	retry state exhausted	after stock flash, set_active; if R1 still stuck, use documented boot_para/para reset
UBports installer cannot auto-detect	ADB/device detection	device not in Android/ADB or unusual USB state	select rabbit r1 manually; enter bootloader when requested
UBports install boots recovery but system-image install fails	recovery/install layer	installation payload or userdata/log issue	save installer/recovery logs; retry supported path, not r1_escape custom vbmeta

24. Command reference

24.1 Read-only fastboot state capture

```
fastboot devices -l
fastboot getvar current-slot
fastboot getvar is-userspace
fastboot getvar slot-successful:a
fastboot getvar slot-unbootable:a
fastboot getvar slot-retry-count:a
fastboot getvar slot-successful:b
fastboot getvar slot-unbootable:b
fastboot getvar slot-retry-count:b
fastboot flashing get_unlock_ability
fastboot getvar all 2>&1 | tee fastboot-all.txt
```

24.2 Proven R1 preloader -> bootloader-fastboot entry

```
python3 r1_escape/mtkbootcmd.py FASTBOOT
fastboot devices -l
```

24.3 Stock AVB diagnostic principle

For a clean stock test:

```
YES: stock vbmeta images, flashed normally, bootloader remains unlocked
NO:  --disable-verity / --disable-verification
NO:  relocking the bootloader
NO:  mixing r1_escape vbmeta with stock boot/super
```

24.4 Slot activation

```
fastboot set_active a
```

24.5 R1-specific deeper slot reset only when needed

```
fastboot erase boot_para
fastboot erase para
fastboot reboot bootloader
```

24.6 Backup raw security/config partitions before last-resort work

```
python3 mtk r seccfg /tmp/r1-seccfg-before.bin
python3 mtk r frp    /tmp/r1-frp-before.bin
sha256sum /tmp/r1-seccfg-before.bin /tmp/r1-frp-before.bin
```

25. Glossary

ADB Android Debug Bridge. A userspace debugging/communication protocol. Different from fastboot.

A/B slots Two bootable sets of slot-aware Android partitions, normally a and b, used for seamless updates and rollback.

AVB Android Verified Boot 2.0. The signed metadata and verification framework used by modern Android.

BROM MediaTek Boot ROM: immutable first-stage code in the SoC.

boot.img Android boot image containing the Linux kernel and early userspace/ramdisk components.

bootloader fastboot Fastboot implemented in the bootloader. Available before Android/recovery userspace.

dm-verity Linux device-mapper integrity target used to verify large read-only block-device filesystems using hash trees.

FRP Factory Reset Protection partition. In the `r1_escape` flow, a byte in this partition is used to alter the R1's bootloader-unlock permission state.

fastbootd Userspace fastboot, normally entered through recovery. Designed to manipulate Android dynamic logical partitions.

GSI Generic System Image. Android's Treble architecture and Halium both use generic system-image concepts to separate generic OS code from vendor hardware support.

Halium Hardware abstraction project that lets GNU/Linux mobile systems reuse Android kernels, vendor services, and blobs. Ubuntu Touch Android-device ports use it.

preloader MediaTek vendor boot stage after BROM. On your R1 it exposes a CDC ACM serial interface that `mtkbootcmd.py` uses.

`seccfg` MediaTek security/configuration partition. The R1 community de-verity recovery found persistent lock/managed-verity state here.

`super` Physical Android dynamic-partition container holding logical partitions such as `system`, `vendor`, and `product`.

`vbmeta` AVB metadata image containing or chaining the signed descriptors used to verify partitions.

26. Sources and further reading

The technical model and installation path in this document were cross-checked against current upstream/vendor/community material on August 7, 2026.

Final checklist for your exact situation

Before your next write operation, answer these in order:

1. Can I enter bootloader fastboot? YES - your logs proved this.
2. Is the bootloader unlocked? It was; verify again.
3. What is the current slot? Inspect now.
4. Are either/both slots marked unbootable? Inspect now.
5. Have I restored one coherent stock v0.8 set? Not yet, based on the supplied log.
6. Does clean stock boot without dm-verity? Test after coherent restore.
7. Only if no: is managed verity=eio persistent? Capture evidence.
8. Only if yes: use the seccfg recovery path.
9. Once stock boots: run UBports Installer.
10. Do I need fastbootd first for Ubuntu Touch? NO.

The single most useful change in approach is to stop treating each error as a new command to try. Treat the device as a chain of stateful layers, restore each layer to one known configuration, and only move downward into raw MediaTek state when the higher-level stock test proves it is necessary.

Reference notes

- [1] UBports, **rabbit r1 device page**, <https://devices.ubuntu-touch.io/device/r1/>
- [2] UBports, **Installer configuration for `r1`**, <https://github.com/ubports/installer-configs/blob/master/v2/devices/r1.yml>
- [3] Jonathan Procter, **Rabbit R1 Recovery Notes: Bad State, dm-verity/de-verity Corruption, Boot Loops, and Factory Restore**, <https://github.com/jonathanprocter/rabbit-r1-deverity-recovery/blob/main/docs/rabbit-r1-de-verity-recovery-community-writeup.md>
- [4] UBports, **Ubuntu Touch 24.04-1.0 release**, <https://ubports.com/blog/ubports-news-1/ubuntu-touch-24-04-1-0-release-3973>
- [5] Android Open Source Project, **Dynamic partitions**, https://source.android.com/docs/core/ota/dynamic_partitions
- [6] Android Open Source Project, **Android Verified Boot**, <https://source.android.com/docs/security/features/verifiedboot/avb>
- [7] Android Open Source Project, **Use Verified Boot**, <https://source.android.com/docs/security/features/verifiedboot/verified-boot>
- [8] Android Open Source Project, **Implement dm-verity**, <https://source.android.com/docs/security/features/verifiedboot/dm-verity>
- [9] Android Open Source Project, **Implement A/B updates**, https://source.android.com/docs/core/ota/ab/ab_implement
- [10] Android Open Source Project, **Implement OTA updates / A/B bootloader interactions**, <https://source.android.com/docs/core/architecture/bootloader/updating>
- [11] Android Open Source Project, **Move fastboot to userspace**, <https://source.android.com/docs/core/architecture/bootloader/fastbootd>
- [12] RabbitHoleEscapeR1, **r1_escape / r1.sh**, https://github.com/RabbitHoleEscapeR1/r1_escape/blob/main/r1.sh
- [13] Android Open Source Project source tree, fastboot build packaging includes host filesystem tools such as mke2fs, <https://android.googlesource.com/platform/system/core/>
- [14] UBports device infrastructure, **rabbit r1 metadata**, <https://gitlab.com/ubports/infrastructure/devices.ubuntu-touch.io/-/blob/main/data/devices/r1/data.md>
- [15] UBports documentation, **Introduction to porting**, <https://docs.ubports.com/en/latest/porting/introduction/Intro.html>
- [16] UBports documentation, **Run desktop applications / Libertine**, <https://docs.ubports.com/en/latest/userguide/dailyuse/libertine.html>
- [17] UBports documentation, **Install Ubuntu Touch**, <https://docs.ubports.com/en/latest/userguide/install.html>